

# POSTMORTEM

## **Overview**

This assignment was an individual task requiring me to create a videogame with a given framework from scratch, the development process of the game was at although times difficult, informative and entertaining at some points during the development process. I decided to name the game SKRT, and intended for the project to be a “roguelike” with a twist that the player character would be dealing the damage through collisions, rather than with bullets or projectiles like other games in the genre.

Through this assignment, I learned the basics of planning and creating a game, with the creation of a game pitch document as well as a technical design document that would be useful reference for informing me on how I wanted to implement specific features of my game. Furthermore, the debugging tools and instructions provided to us helped with the locating and deleting of errors such as memory leaks and tracing improperly implemented libraries and code.

The game was written in C++ and served as a good refresher on how the language functioned as well as the standards expected when coding a project as large as this. Additionally with the framework we were given, as well as the implementation of middleware such as FMOD and Box2D, I was able to create a responsive (although somewhat rough around the edges) game that represented the idea I had in terms of gameplay and art design.

## **What Went Right**

I think the general gameplay of SKRT properly represents the idea I had in mind, containing an upgrade system and levelling system, a director that can spawn enemies, as well as 4 different (although somewhat rudimentary) enemies that would spawn overtime.

I feel that the movement system although simple works well for the game’s favour, requiring the user to use their time quickly to dodge and get out of the way of enemies without giving them too much breathing room that it becomes too simple. This is emphasized further by the “Boss” enemy that will attempt to charge into the player, punishing them for moving too slow or getting too close.

I believe the art style I went for would help the game stand out from other styles, using UNICODE symbols to create the sprites through photobashing them together, as well as keeping the style in black and white to help certain elements of the game stand out (such as experience, the charge up meter of the character, as well as the boss).

I believe that the enemies added to the game provide their own unique challenges, (not included the demon, being a basic follower/fodder enemy). For example, the minelayer planting additional obstacles around the map that the player can either interrupt or have to deal with dodging if left unchecked, or the slower, which punishes the player for not getting outside its range fast enough by cutting their speed. I think the animations used also helped the player recognize that certain enemies were doing special functions that distinguished them.

## ***What Went Wrong***

Though the game was technically complete, I feel that it could have used more polish in some areas that I would not have enough time to complete, e.g. more upgrades, finished special upgrades system, additional art polish etc. I hope with the shared workload of a group project assignment coming up next, I will be able to properly polish out features in the game.

Initially, I thought I would leave handling memory leaks via deletes and assignments to 0 once I had completed the basic framework of the game. This was an error, as I failed to keep track of all pointer based objects and the order they would be deleted, leading me needing to spend extra time tracking down every pointer and object as well as how they would be deleted and what.

Some issues in the input system I was unable to fix, for example pressed and held would seem to be a bit buggy, as well as the mouse sometimes being dragged off the screen, causing the game to think the user was still holding click mid game and disrupting the flow of the experience.

With my limited knowledge of FMOD, sometimes the sound system tends to bug out, this is especially apparent when the player is surrounded by several enemies, in which the system will repeatedly spam the hit sound at loud volumes. Unfortunately I wasn't able to wrap my head around the documentation of the system and was unable to fix it.

With the implementation of the enemybase, I feel like it was too hardcoded and wasn't given enough functions, leading me to need to rewrite practically all of it for future enemy ai behaviours due to the chain of dependencies between variables stored in them, which led to some time that could've been spent better.

## ***Lessons Learnt***

For the future, I'll make sure that any new object created will always delete itself fully without leaving any additional memory that becomes wasted rather than saving it to be fixed later. This will greatly help in saving resources and time and can allow me to work on other more important parts of the game.

This assignment taught me how to properly format subclasses, as well as how to implement functions in subclasses in order to reduce the need to repeatedly code the same functions, as well as when I should override them to suit a new object's behaviour. Additionally, the assignment taught me how to go about coding AI for enemy behaviour, although next time I would like to try a new system of formatting different behaviours, as my current code is rather messy and somewhat bashed together rather than flowing neatly.

In my next game, I'll need to make sure to plan out my game's feature in greater detail, as it sort of branched off of me working on one set of things to another (e.g. upgrades and then a director and then a pause menu led me to become a bit confused when returning to the upgrade system to fix it up).

With a basic idea of what a game needs, hopefully this means I can set up the basic systems of a game for next time in a more modular manner, allowing me to easily add or remove features without needing to code so much of it manually for functions that did very similar things (specifically behaviour such as following a target or how an enemy should attack when it needs to).

## Conclusion

To conclude, this assignment taught me how to firstly create a game for at least a demo form, as well as what's required in terms of backend planning and coding, as well as the smaller things that require consideration such as memory leak tracking or smaller values that could potentially break the game (see below of early build in which the director didn't have a proper spawn timer so it would over spawn demons).

I now understand how important the implementation of a modular system is in creating a videogame, as though I had a simple modular system in the form of upgrade templates, this could've been further extended to the creation of new enemies for easier coding and I will ensure that I follow a better coding standard in terms of superclasses and subclasses.

In the end, I believe that coding videogames is a rather fulfilling process, as though it may take a while, I find when everything works together in the manner intended, it is a greatly rewarding and satisfying process, and I hope to be able to work on additional videogames in the future. Though it is still pretty funny when unexpected things occur too (again refer to below image).

